

UNIT – II

DESIGN AND ANALYSIS OF ALGORITHMS

DISJOINT SETS

DISJOINT SET OPERATIONS

- **Definition** : Sets are represented as a collection of well-defined objects or elements and it does not change from person to person. A set is represented by a capital letter symbol and the number of elements in the finite set is represented as the cardinal number of a set.
- **Disjoint Sets** : If two sets are A and B and there is no element that is in both A and B then both these sets are called disjoint each other.
- **Ex:** $A = \{1, 3, 5, 9\}$ and $B = \{2, 4, 6, 8\}$ are disjoint sets.

Set Operations :

- **Union** : The union of sets A and B denoted by $A \cup B$ is the set defined as

$$A \cup B = \{x / x \in A \text{ or } x \in B\}$$

$$A = \{1, 2, 3, 4\} \text{ and } B = \{1, 3, 5, 6\}$$

$$A \cup B = \{1, 2, 3, 4, 5, 6\}$$

- **Intersection** : The intersection of sets A and B , denoted by $A \cap B$, is the set defined as $A \cap B = \{x / x \in A \text{ and } x \in B\}$

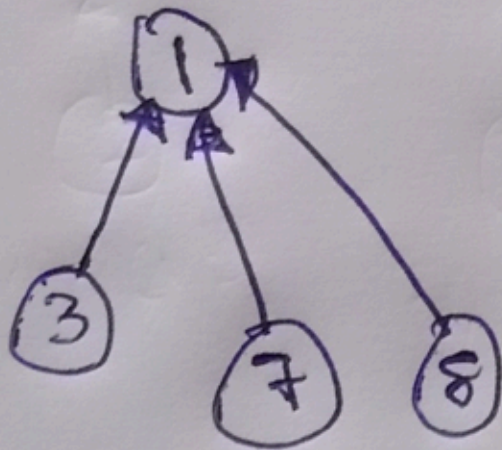
$$A = \{1, 2, 3, 4\} \text{ and } B = \{1, 3, 5, 6\}$$

DISJOINT SET OPERATIONS

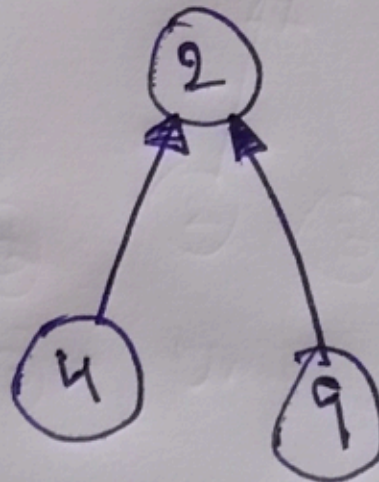
- **Find(i)** : Given the element i , find the set containing i . The element 4 is in A and 5 is in B .
- **Difference** : The difference of sets A and B (A from B) denoted by $A - B$ is the set defined as $A - B = \{ x / x \in A \text{ and } x \notin B \}$
- **Complement** : For a set A , the difference $\mu - A$, where μ is the universal set, is called the complement of A and it is denoted by $\bar{A}$.
Thus $\bar{A}$ is the set of everything that is not in A .
- **Cartesian Product** : Let $A_1, A_2, \dots, A_n$ be n sets, then the set of all ordered n -tuple $\langle x_1, x_2, \dots, x_n \rangle$, where $x_i \in A_i$ for all i , $1 \leq i \leq n$ is called the Cartesian product of $A_1, A_2, \dots, A_n$ and is denoted by $A_1 \times A_2 \times \dots \times A_n$
- **Equality of n tuples** : The ordered tuples $\langle x_1, x_2, \dots, x_n \rangle$ and $\langle y_1, y_2, \dots, y_n \rangle$ are equal if $x_i = y_i$ for all i , $1 \leq i \leq n$. Ex: $\langle 1, 2, 3 \rangle \neq \langle 3, 2, 1 \rangle$

Sets Representation

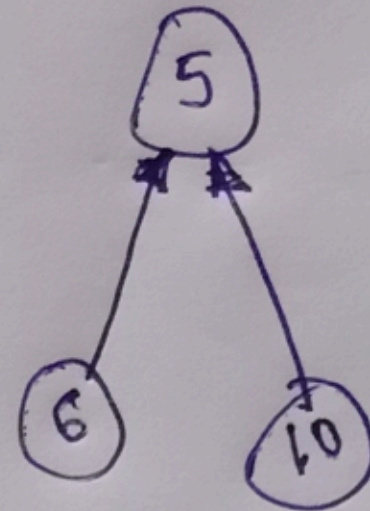
$$S_1 = \{1, 3, 7, 8\} \quad S_2 = \{2, 4, 9\} \quad S_3 = \{5, 6, 10\}$$



S_1

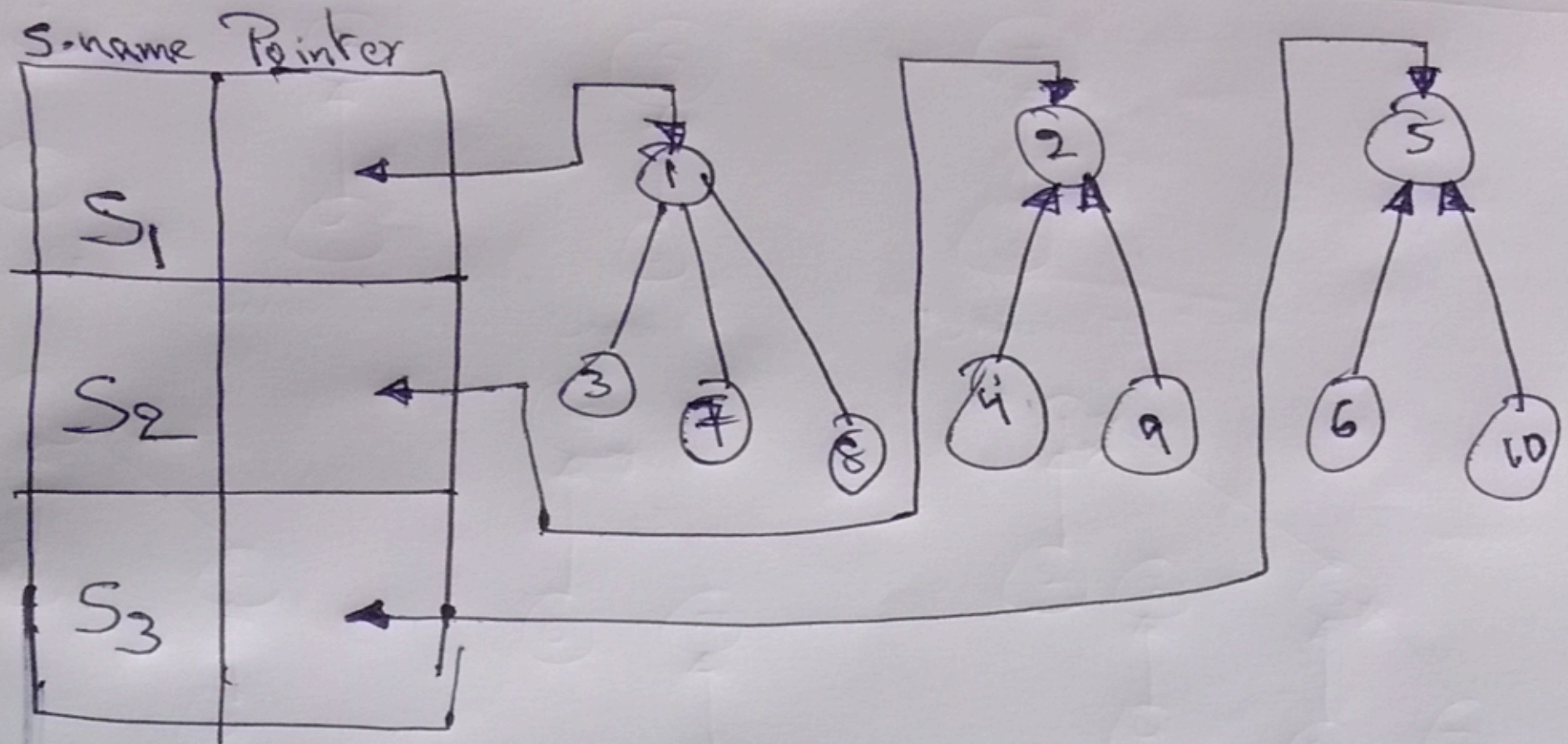


S_2



S_3

Data Representation



FindPointer(S₁)

FindPointer(S₂)

FindPointer(S₃)

DISJOINT SET OPERATIONS

- **UNION** : To obtain the Union of two sets, set the parent field of one of the roots to the other root.
- This can be accomplished easily, if with each set name, we keep a pointer to the root of the tree representing that set.
- Each root has a pointer to the set name, then to determine what set an element is currently in. We follow parent link to the root of its tree and use the pointer to the set name.
- If we determine that the element i is in a tree with root j , and j has a pointer to k in the set name table, then the set name is just $\text{name}[k]$.
- If we wish to unite sets S_i and S_j , then we wish to unite the sets with roots $\text{FindPointer}(S_i)$ and $\text{FindPointer}(S_j)$. Here $\text{FindPointer}()$ is a function that takes a set name and determine the root of the tree that represent it.
- In many applications the set name is the element at the root.
- The function $\text{Union}(i, j)$ requires two trees with roots i and j be joined.

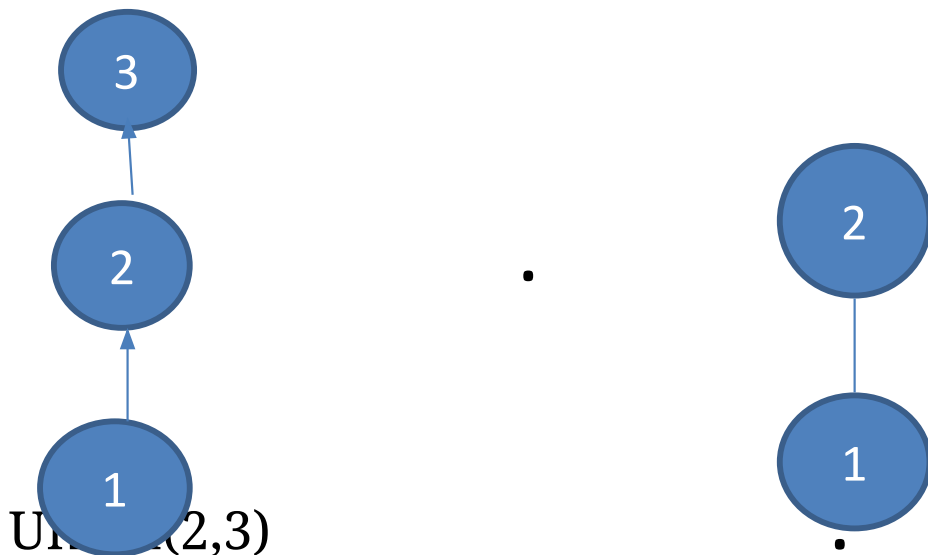
DISJOINT SET OPERATIONS

- Union(i, j) is very simple. We pass in two trees with roots i and j . Adopting the convention that the first tree become a subtree of the second. The statement $p[i] = j$ accomplishes the union.
- Algorithm SimpleUnion(i, j)
 {
 $p[i] := j$;
 }
- Algorithm SimpleFind(i)
 {
 while($p[i] > 0$) do
 $i := p[i]$;

DISJOINT SET OPERATIONS

- Now process the sequence of Union-Find operations for n elements : Union(1, 2), Union(2, 3), Union(3, 4) Union($n-1$, n). And Find(1), Find(2) Find(n).
- The sequence results in the final tree structure as follows:

Union(1, 2)



DISJOINT SET OPERATIONS

- n-1 union operations are required to construct the above tree structure.
- Since the time required to process a find for an element at level i is $O(i)$.
- The total time needed to process the n finds is

$$O(\sum_{i=1}^n i) = O(n^2)$$

- We can improve the performance by using **Weighting Rule**.

Weighting Rule : If the number of nodes in the tree with root i is less than the number of nodes in the tree with root j, then make j the parent of i, otherwise make i the parent of j.

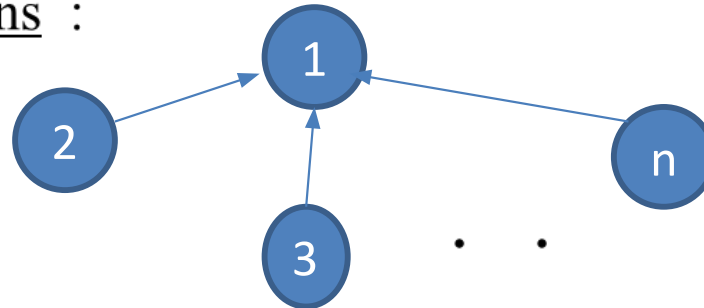
- Now process the sequence of Union-Find operations for n elements : Union(1, 2), Union(1, 3), Union(1, 4) Union(1, n). And Find(1), Find(2) Find(n).

The final tree structure is as follows:

Initially :



After performing n-1 Unions :



DISJOINT SET OPERATIONS

- We maintain **count** field to maintain number of nodes in each tree. If i is the root node, then $\text{count}(i)$ equals the number of nodes in that tree. All the nodes other than root node have +ve number in the P field. We can maintain the count in P field of the root as a -ve number.
- The time required to perform union by below algorithm has increased somewhat but still bounded by constant .

Algorithm WeightedUnion(i, j)

```

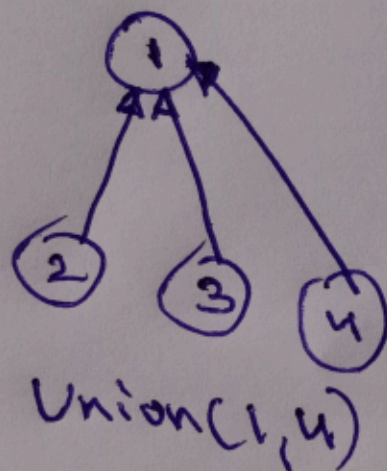
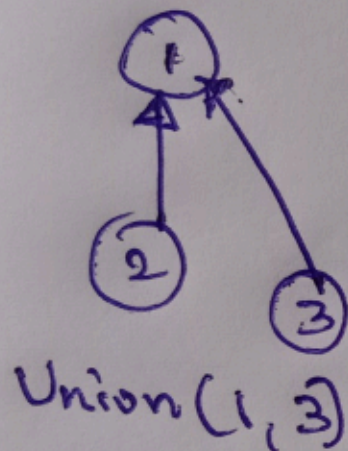
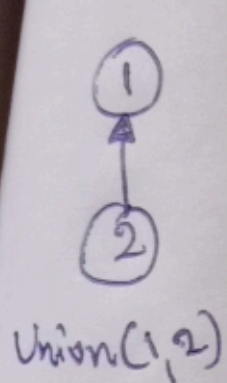
{
    temp := P[i] + P[j]
    if(P[i] > P[j])
    {
        P[i] := j;
        P[j] := temp;
    }
    else
    {
        P[j] := i;
        P[i] := temp;
    }
}

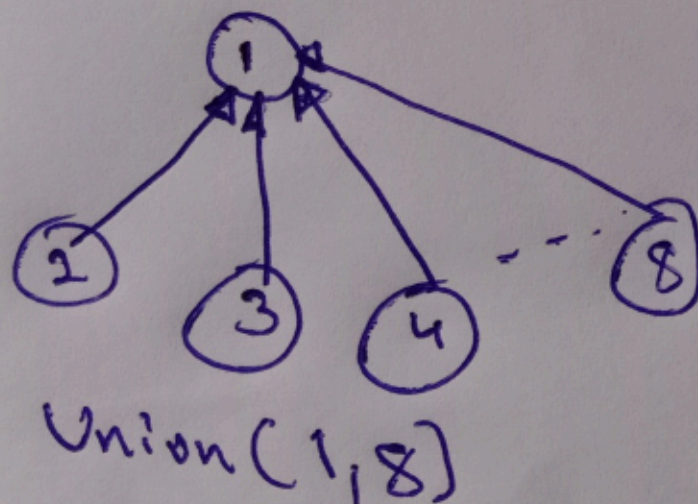
```

- The height of tree with m nodes is no greater than $\lfloor \log^m \rfloor + 1$. so the maximum time needed to perform find operation is the same as $\lfloor \log^m \rfloor + 1$

Weighted Union

① ② ③ ④ ⑤ ⑥ ⑦ ⑧







Union(1, 2)



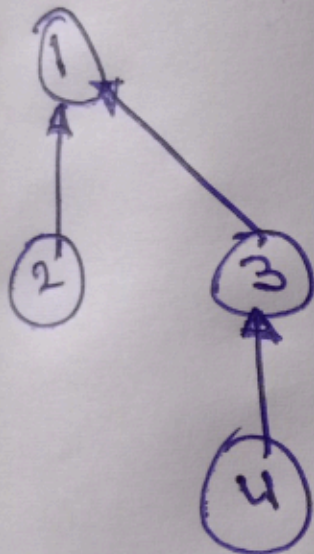
Union(3, 4)



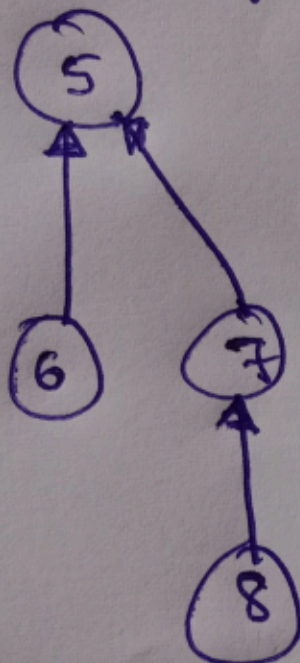
Union(5, 6)



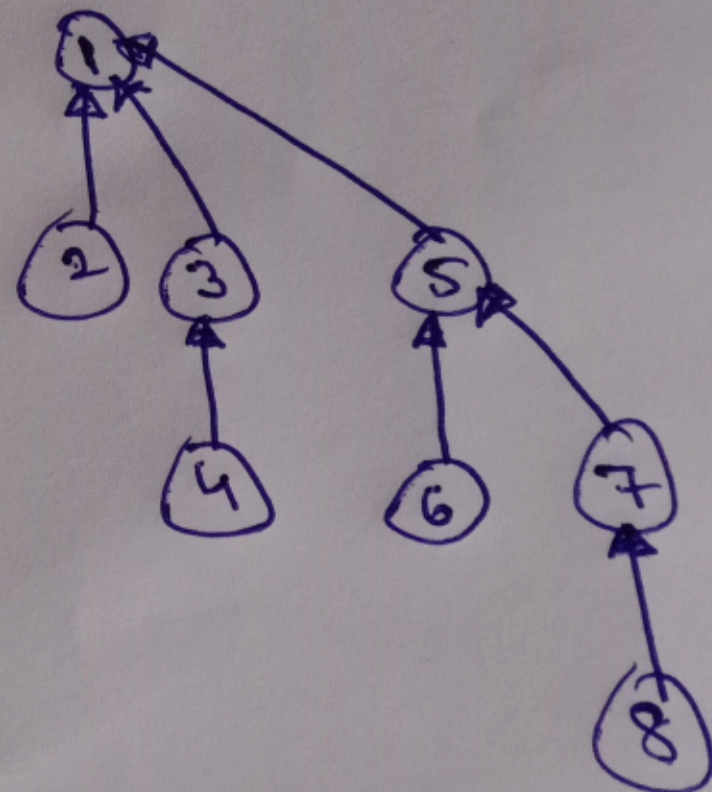
Union(7, 8)



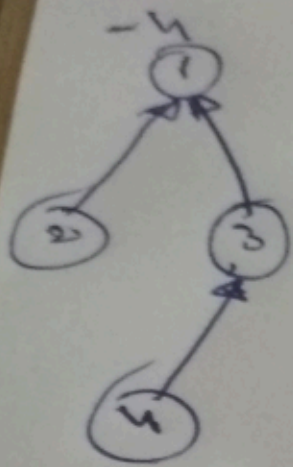
Union(1, 3)



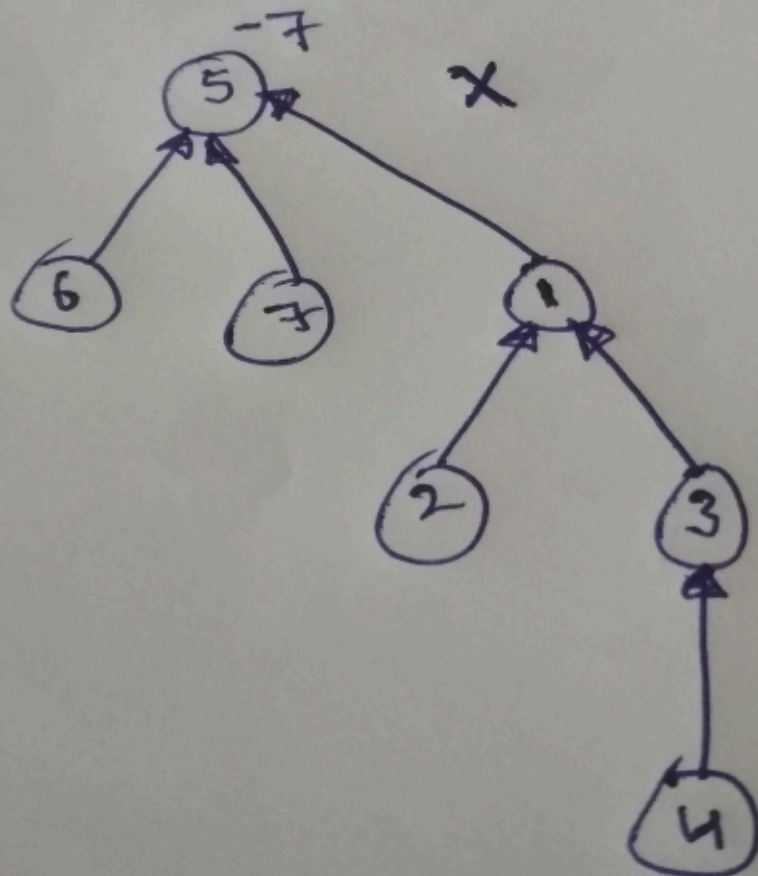
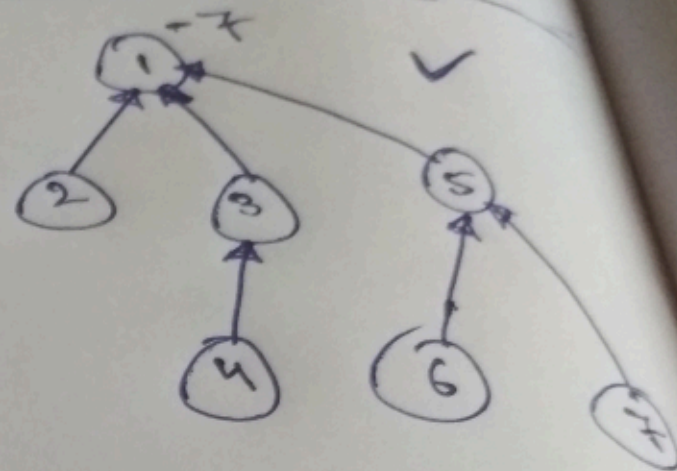
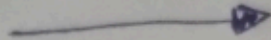
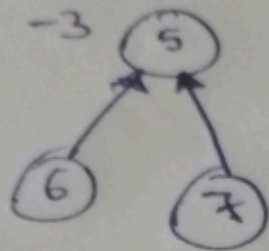
~~Union(4, 5)~~
Union(5, 7)



Union(1, 5)



+



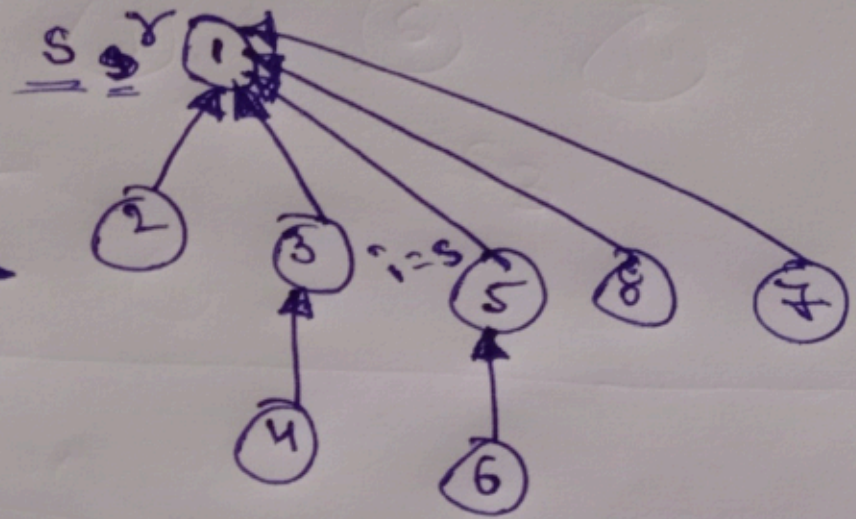
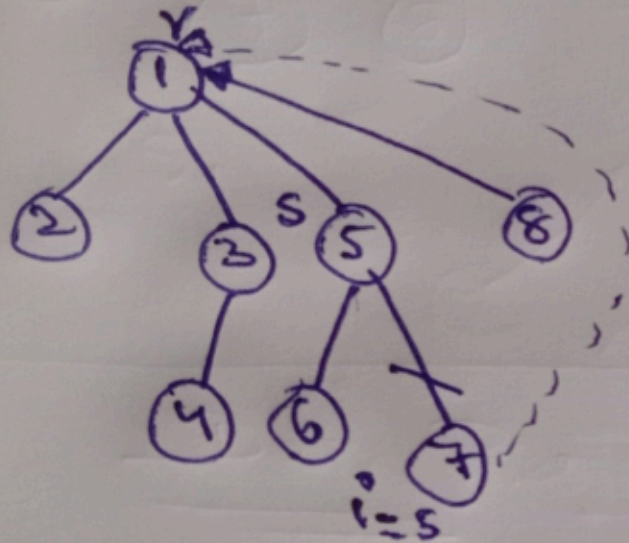
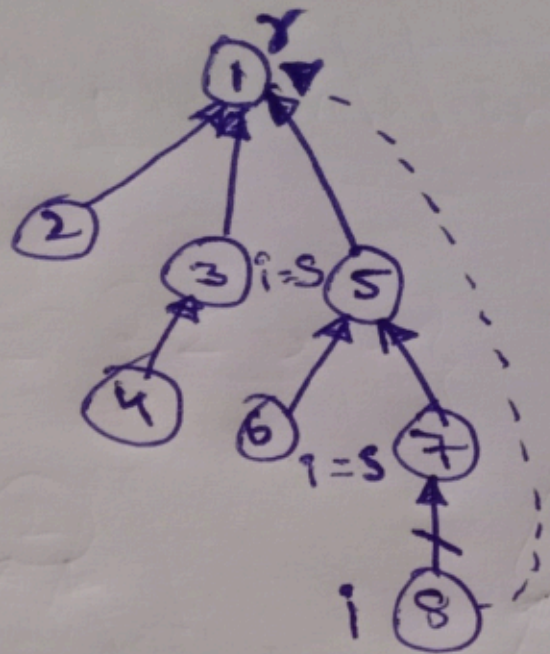
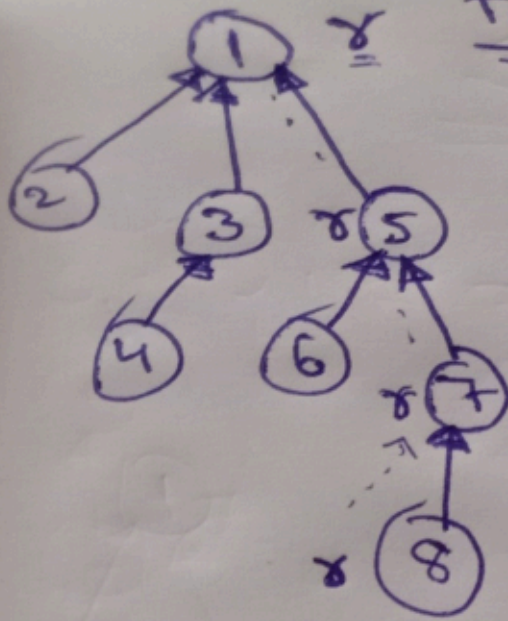
DISJOINT SET OPERATIONS

- **Collapsing Rule for Find** : if j is a node in the path from i to its root and $P[j] \neq \text{root}[j]$, then set $P[j]$ to the $\text{root}[j]$.

Algorithm CollapsingFind(i)

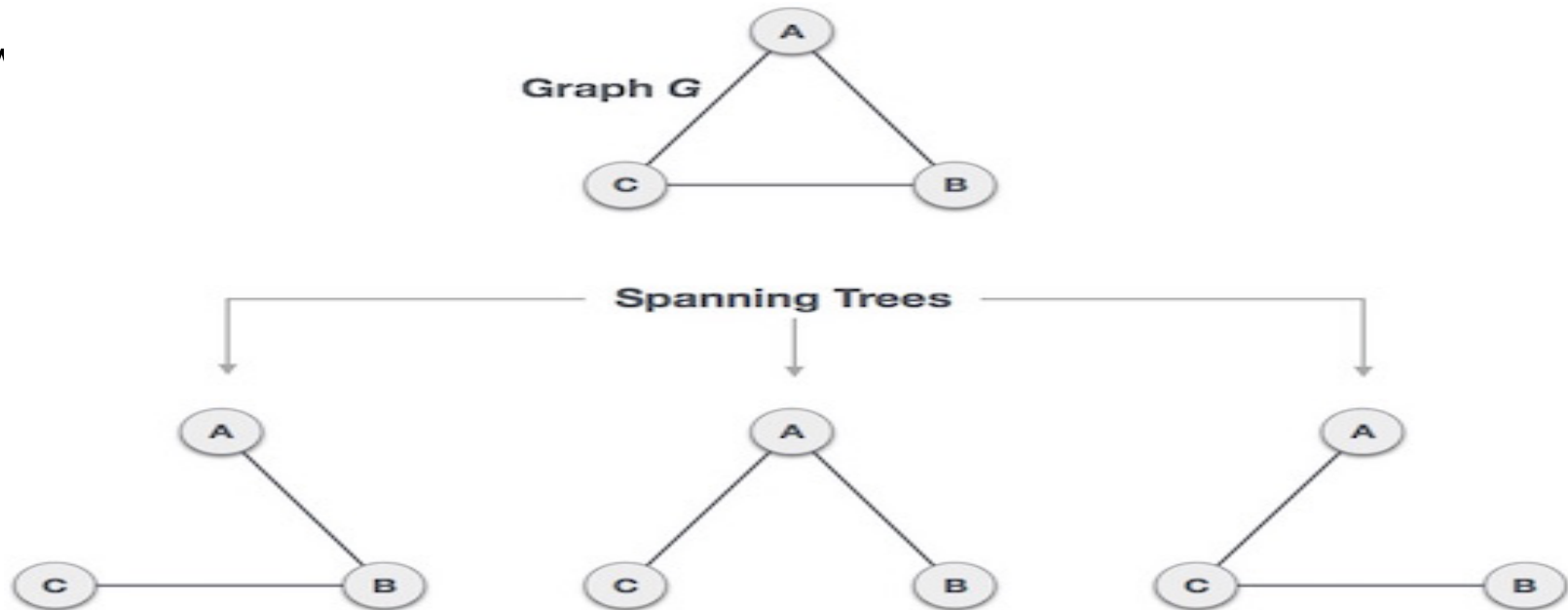
```
{
    r := i;
    while( P[r] > 0) do
        r := P[r];
        while(i ≠ r) do
        {
            s := P[i];
            P[i] := r;
            i := s;
        }
    return r;
}
```

Find(8)



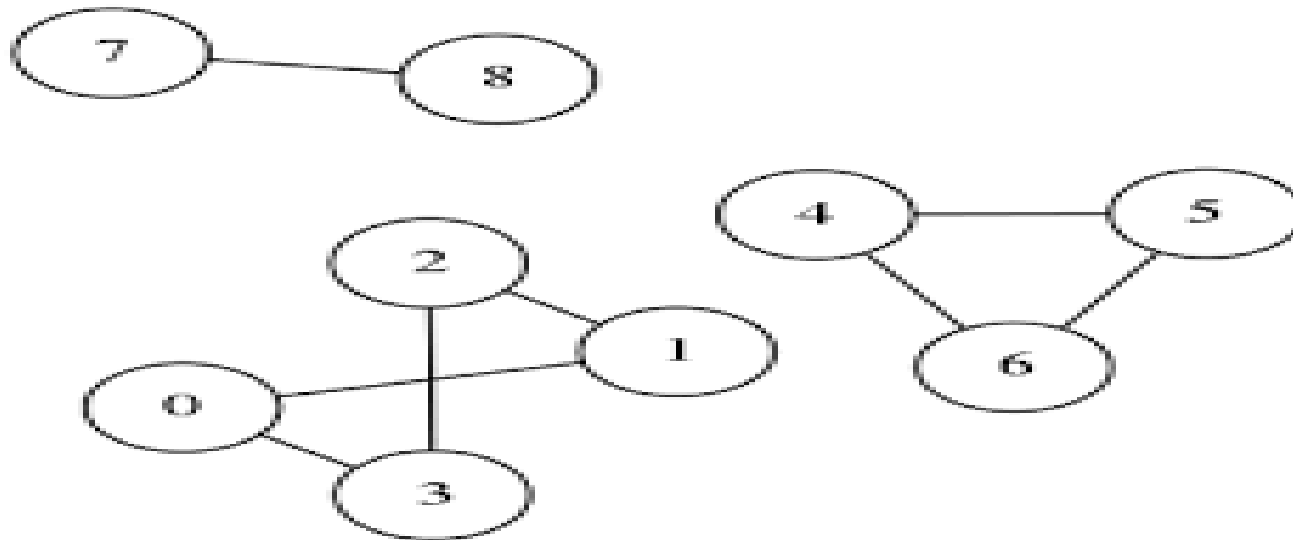
SPANNING TREES

- **Definition** : Let $G=(V, E)$ be an undirected connected graph. A sub-graph $t=(V,E')$ of G is a spanning tree of G if t is a tree.
- A spanning tree is a subset of Graph G , which has all the vertices covered with minimum possible number of edges. Hence, a spanning tree does not have cycles and it cannot be disconnected.
- we can draw a conclusion that every connected and undirected Graph G has at least one spanning tree. A disconnected graph does not have any spanning tree, as it cannot be spanned to all its vertices.
- A complete undirected graph can have maximum n^{n-2} number of spanning trees, v



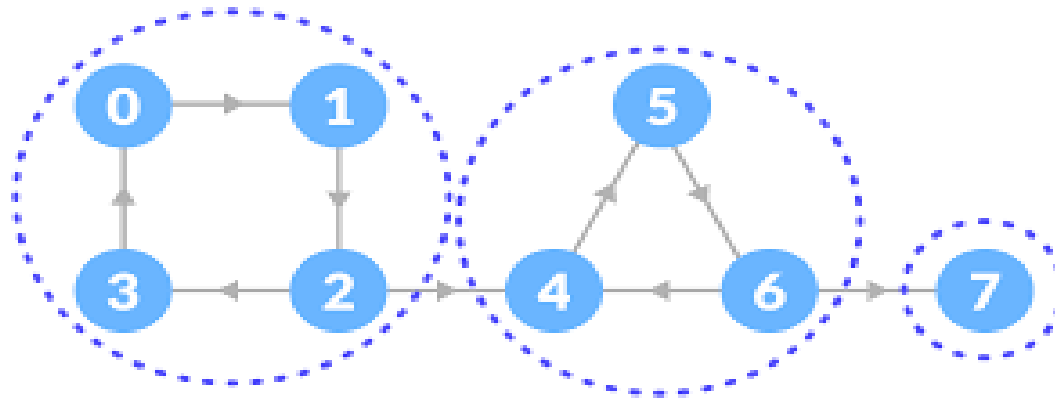
CONNECTED COMPONENTS

- **Definition** : A connected component of an undirected graph is a maximal set of nodes such that each pair of nodes is connected by a path.
- Connected components form a **partition** of the set of graph vertices, meaning that connected components are non-empty, they are pairwise disjoint, and the union of connected components forms the set of all vertices.
- Equivalently, we can say that the relation that associates two nodes if and only if they belong to the same connected component is an **equivalence relation** that is it is reflexive, symmetric, and transitive.



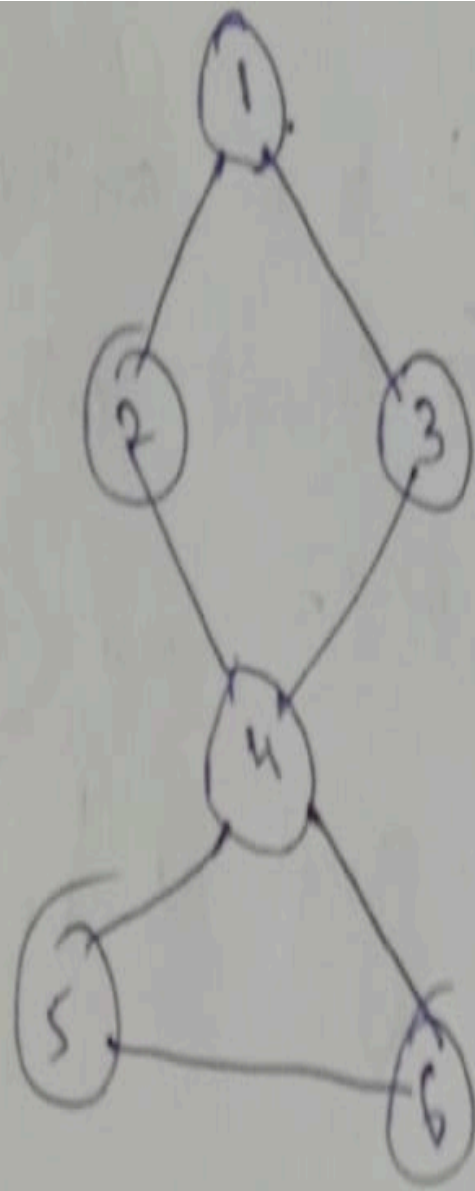
CONNECTED COMPONENTS

- **Strongly Connected Components** : Strong Connectivity applies only to directed graphs. A directed graph is strongly connected if there is a directed path from any vertex to every other vertex. This is same as connectivity in an undirected graph, the only difference being strong connectivity applies to directed graphs and there should be directed paths instead of just paths. Similar to connected components, a directed graph can be broken down into Strongly Connected Components.



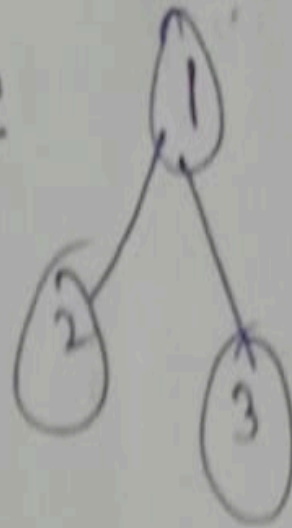
Bi- CONNECTED COMPONENT

- Definition : A **biconnected component** of a graph is a connected subgraph that cannot be broken into disconnected pieces by deleting any single node (and its incident links). An *articulation point* is a node of a graph whose removal would cause an increase in the number of connected components. Articulation points can be important when you analyse any graph that represents a communications network.
- A **bi-connected component** (sometimes known as a **2-connected component**) is a maximal bi-connected sub-graph. Any connected graph decomposes into a tree of bi-connected components called the **block-cut tree** of the graph. The blocks are attached to each other at shared vertices called **cut vertices** or **articulation points**. Specifically, a **cut vertex** is any vertex whose removal increases the number of connected components.
- A graph is said to be Biconnected if:
 - It is connected, i.e. it is possible to reach every vertex from every other vertex, by a simple path.
 - Even after removing any vertex the graph remains connected.
- A bi-connected graph is a connected and “Nonseperable” graph.

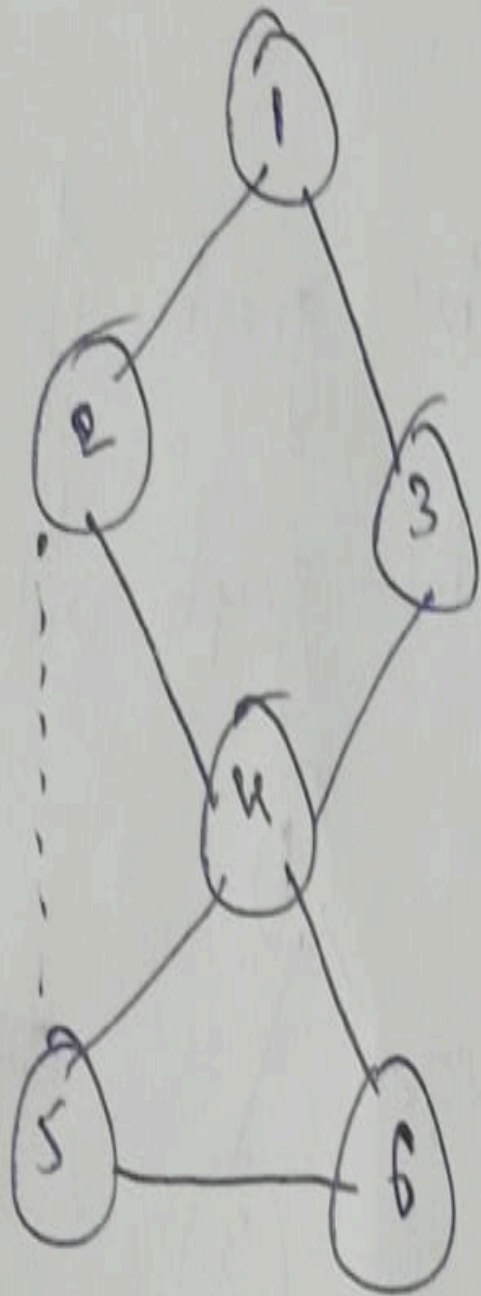


In this graph if we remove 4
the graph will be disconnected

as:

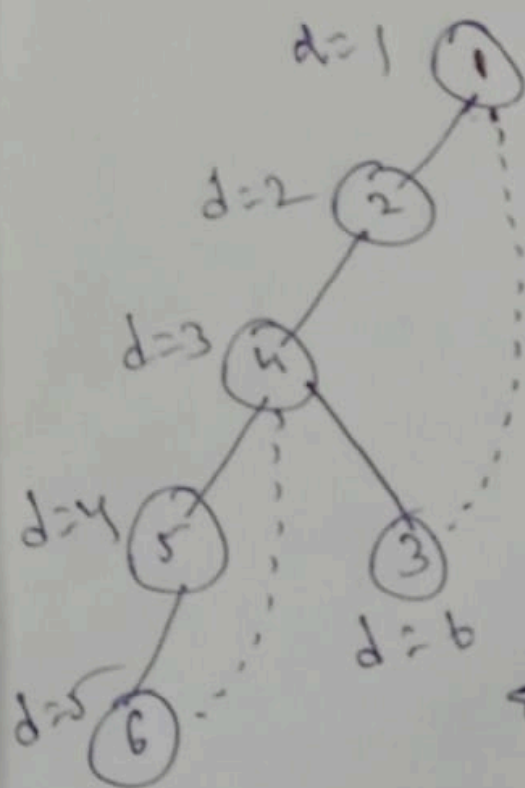


So the node 4 is an articulation point.



It is an articulation point
So connect either (2) and (5)
or (3) and (6) to make
the graph as bi-connected.

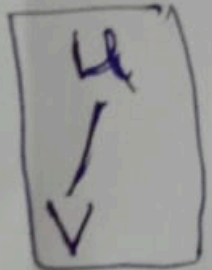
Construct Depth First Search Tree



	1	2	3	4	5	6
d :	1	2	6	3	4	5
L :	1	1	1	1	4	4

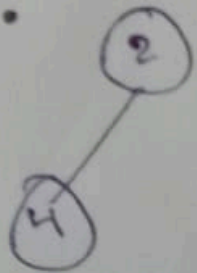
$$\text{if } L[v] \geq d[u]$$

$u \rightarrow \text{Parent}$
 $v \rightarrow \text{child}$



then u is an articulation point

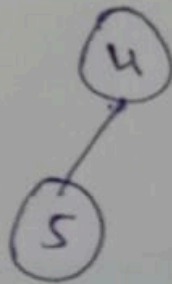
Except root node, Consider all the remaining nodes



$$L[u] \geq d[v]$$

$$1 \geq 2$$

False



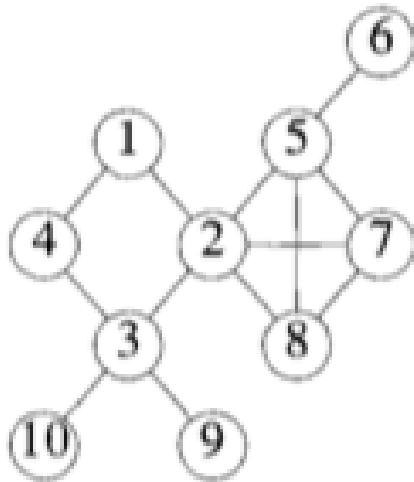
$$L[u] \geq d[v]$$

$$4 \geq 3$$

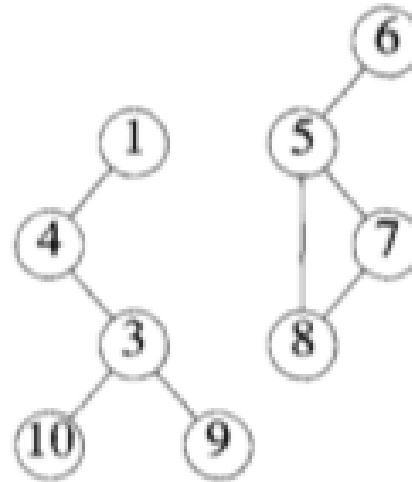
Yes

so u is an articulation point

EXAMPLE:



(a) Graph G



(b) Result of deleting vertex 2

DIVIDE AND CONQUER

GENERAL METHOD

- Given a function to compute on n -inputs the Divide-and-Conquer(DAC) strategy suggests splitting the input into k distinct subsets, $1 < k \leq n$, yielding k sub-problems. These sub-problems must be solved, and then the method must be found to combine solutions into a solution of a whole problem.
- The below algorithm is a general method of Divide-and-Conquer.

Algorithm DandC(P)

{

GENERAL METHOD

- Combine is the function that determines the solution to P using the solution to the k sub-problems.
- The computing time of DandC is described by the following recurrence relation:

$$T(n) = \begin{cases} g(n) & \text{if } n \text{ is small} \\ T(n_1) + T(n_2) + \dots + T(n_k) + f(n) & \text{other} \end{cases}$$

- Where $g(n)$ is the time to compute the answer directly without splitting and $T(n)$ is the time for DandC on any input size n .
- The computing time of DandC is described by the following recurrence relation:

$$T(n) = \begin{cases} T(1) & \text{if } n=1 \\ aT(n/b) + f(n) & \text{if } n>1 \end{cases}$$

- Where a and b are known constants

GENERAL METHOD

- We assume that $T(1)$ is known and n is power of b ($n=b^k$)
- One of the methods to solving any such recurrence relation is called **the “Substitution Method”**
- **Ex : Consider the case in which $a = 2$ and $b = 2$. Let $T(1) = 2$ and $f(n) = n$.**

$$\begin{aligned}T(n) &= 2 T(n/2) + n \\&= 2 [2T(n/4) + n/2] + n \\&= 4 T(n/4) + 2n \\&= 4 [2T(n/8) + n/4] + 2n \\&= 8 T(n/8) + 3n \\&= 2^3 T(n/2^3) + 3n \\&\quad \cdot \\&\quad \cdot \\&\quad \cdot \\&= 2^i T(n/2^i) + i n\end{aligned}$$

$$n = 2^i \text{ so } i = \log_2 n$$

$$T(n) = 2^{\log_2 n} T(n/2^{\log_2 n}) + n 2^{\log_2 n}$$

$$T(n) = n T(1) + n \log_2 n$$

$$\mathbf{T(n) = O(n \log_2 n)}$$

APPLICATIONS
OF
DIVIDE – AND - CONQUER

BINARY SEARCH

- Let $A[i]$, $1 \leq i \leq n$, be a list of elements, that are sorted in increasing order. Consider the problem of determining whether the given element x is present in the list.
- If x is present, we determine a value j such that $A[j]=x$. If x is not there in the list then j is set to zero.
- Let $P=(n, A[i], A[i+1], \dots, A[l], x)$ denote an arbitrary instance of this search problem.
- Let $\text{Small}(P)$ be true if $n=1$. In this case $S(P)$ will take the value i if $A[i]=x$, otherwise it will take the value zero.
- If P has more than one element, it can be divided into new sub-problems as: pick an index q in the range $[i, l]$ and compare x with $A[q]$, there are three possibilities :
 1. **$X=A[q]$** : in this case the problem P is immediately solved
 2. **$X < A[q]$** : in this case x has to be searched for only in the subset $A[i], A[i+1], \dots, A[q-1]$ so the problem instance is $(q-i, A[i], A[i+1], \dots, A[q-1], x)$.
 3. **$X > A[q]$** : in this case the subset to be searched is $A[q+1], A[q+2], \dots, A[l]$ so the problem instance is $(l-q, A[q+1], A[q+2], \dots, A[l], x)$

BINARY SEARCH

- Binary search is implemented using following steps...
- Step 1 - Read the search element from the user.
- Step 2 - Find the middle element in the sorted list.
- Step 3 - Compare the search element with the middle element in the sorted list.
- Step 4 - If both are matched, then display "Given element is found!!!" and terminate the function.
- Step 5 - If both are not matched, then check whether the search element is smaller or larger than the middle element.
- Step 6 - If the search element is smaller than

Binary Search

Ex: $\overset{1}{2}, \overset{2}{9}, \overset{3}{11}, \overset{4}{15}, \overset{5}{20}, \overset{6}{25}$
A: $\overset{1}{2}, \overset{2}{9}, \overset{3}{11}, \overset{4}{15}, \overset{5}{20}, \overset{6}{25}$

$$\text{low} = 1, \text{high} = 6, \text{mid} = \left\lfloor \frac{1+6}{2} \right\rfloor$$

$$\text{Searching element} = \underline{\underline{25}} \\ x = \underline{\underline{25}}$$

$$A[3] = 11, \boxed{x > A[3]}$$

- so select right side Part of mid

$$\overset{4}{15}, \overset{5}{20}, \overset{6}{25}$$

$$\text{low} = 4, \text{high} = 6, \text{mid} = \left\lfloor \frac{4+6}{2} \right\rfloor = \underline{\underline{5}}$$

$$A[5] = 20, x = 25$$

$$\boxed{x > A[5]}$$

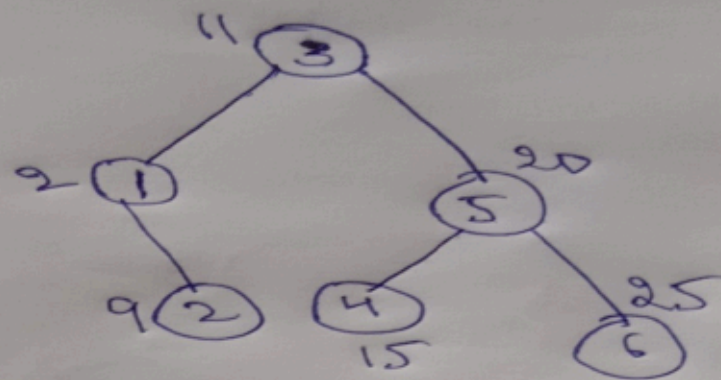
- so select right side portion of mid

$$\overset{6}{25}$$

$$\text{low} = 6, \text{high} = 6, \text{mid} = \left\lfloor \frac{6+6}{2} \right\rfloor = \underline{\underline{6}}$$

$$\boxed{A[6] = x}$$

Search is successful and element is there in 6th location



A: ¹2, ²9, ³11, ⁴15, ⁵20, ⁶25

$$x = \underline{\underline{26}}$$

low	high	mid
1	6	3
4	6	5
6	6	6

$x > A[6]$ - no such
element is there
So un-successful search
— Result is zero

$$x = 15$$

low	high	mid
1	6	3
4	6	5
4	4	4

$$A[4] = \underline{\underline{15}}$$

Search is successful.
it returns "4" as
result.

BINARY SEARCH

Algorithm BinSrch(a, low, high, x)

{

 if(low = high)

 {

 if(x = a[high]) then return high;

 else return 0;

 }

 else

 {

 mid := $\lfloor (\text{low} + \text{high}) / 2 \rfloor$;

 if (x = a[mid]) then return mid;

 else if(x < a[mid]) then

BINARY SEARCH

Complexity

SN	Performance	Complexity
1	Worst case time complexity	$O(\log n)$
2	Best case time complexity	$O(1)$
3	Average Case time complexity	$O(\log n)$
4	Worst case space complexity	$O(1)$

MERGE SORT

- Merge sort is a sorting technique based on divide and conquer technique. With worst-case time complexity being $O(n \log n)$, it is one of the most respected algorithms.
- Merge sort first divides the array into equal halves and then combines them in a sorted manner.
- To understand the following –



- We know that merge sort first divides the whole array iteratively into equal halves unless the atomic values are achieved. We see here that an array of 8 items is divided into two groups of 4 items each.



- This does not change the sequence of appearance of items in the original.



MERGE SORT

- Now, we combine them in exactly the same manner as they were broken down. Please note the color codes given to these lists.
- We first compare the element for each list and then combine them into another list in a sorted manner. We see that 14 and 33 are in sorted positions. We compare 27 and 10 and in the target list of 2 values we put 10 first, followed by 27. We change the order of 19 and 35 whereas 42 and 44 are placed



- In the next values, and sorted order.



- After the final merging, the list should look like this –

Merge Sort

A := ¹3, ²22, ³10, ⁴19, ⁵8, ⁶5

low = 1, high = 6, so $1 < 6$,

$$\text{mid} = \left\lfloor \frac{1+6}{2} \right\rfloor = \left\lfloor \frac{7}{2} \right\rfloor = \underline{\underline{3}}$$

The array A will be divided into two parts

¹3, ²22, ³10 | ⁴19, ⁵8, ⁶5

Consider the above two sub-arrays separately,

- $3^1, 2^2, 10^3$

low = 1 high = 3, $1 < \underline{3} : \text{yes}$

$$\text{mid} = \left\lfloor \frac{1+3}{2} \right\rfloor = \left\lfloor 4/2 \right\rfloor$$

$$= 2$$

- $3^1 \quad 2^2 \quad | \quad 10^3$

consider the left side array

$3^1 \quad 2^2$

low = 1 high = 2, $1 < 2 : \text{yes}$

$$\left\lfloor \frac{1+2}{2} \right\rfloor = \left\lfloor \frac{3}{2} \right\rfloor = 1$$

The above array can be divided
into two sub arrays

$3^1 \quad | \quad 2^2$

The solutions are

$3 \quad | \quad 22 \quad | \quad 10$

$$\begin{array}{ccc}
 & 4 & 5 & 6 \\
 - & 19 & 8 & 5
 \end{array}$$

low = 4, high = 6, $4 < 6$: yes

$$mid = \left\lfloor \frac{4+6}{2} \right\rfloor = \left\lfloor \frac{10}{2} \right\rfloor = \underline{5}$$

The above sub-array can be divided into two parts at 5

$$\begin{array}{ccc}
 4 & 5 & 6 \\
 19 & 8 & 5
 \end{array}$$

- Take left side sub array

$$\begin{array}{cc}
 4 & 5 \\
 19 & 8
 \end{array}$$

low = 4, high = 5

$4 < 5$: yes

$$mid = \left\lfloor \frac{4+5}{2} \right\rfloor = \left\lfloor \frac{9}{2} \right\rfloor = 4$$

So the above array can be divided into two sub parts

$$\begin{array}{cc}
 4 & 5 \\
 19 & 8
 \end{array}$$

The solutions are

$$\begin{array}{ccc}
 19 & 8 & 5
 \end{array}$$

the individual solutions are

merge(1, 2) we get

merge((1, 2), 3) we get

similarly combine left side sub problem solutions

merge(4, 5) produces

now apply merge(4, 5), 6 produces

now we got two sub problem solutions

now apply final merge

Take one temporary array B.

Take two variables i and j. start i at first element in first sub problem and j at first element in the 2nd sub problem.

- if $A[i] \leq A[j]$ then copy $A[i]$ into B and increment i by one.

- otherwise copy $A[j]$ into B and increment j by 1. But for increment B index by

1 commonly.

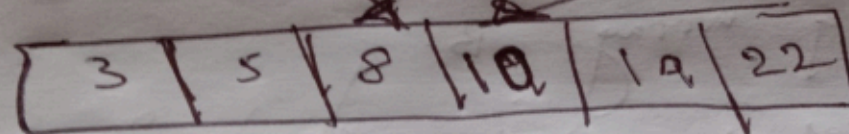
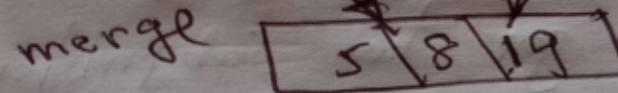
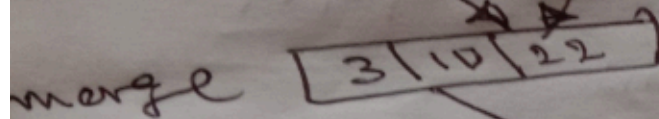
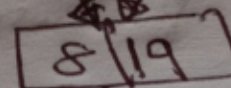
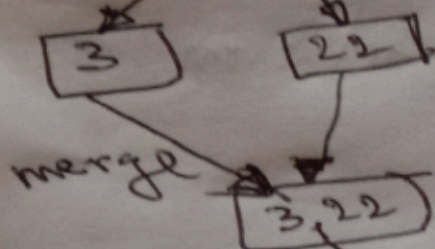
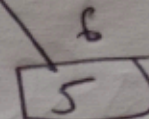
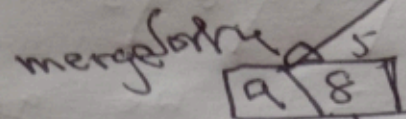
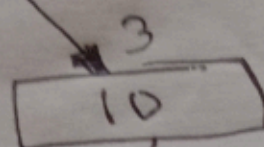
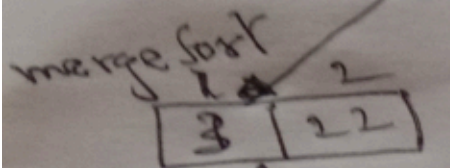
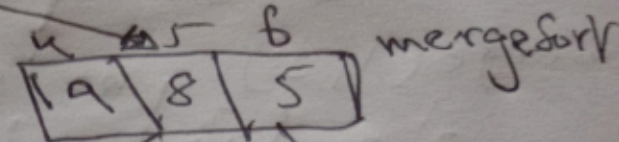
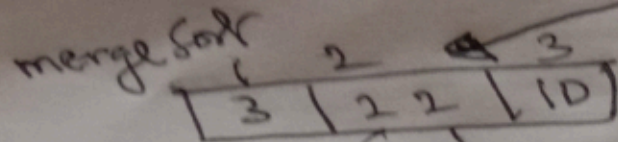
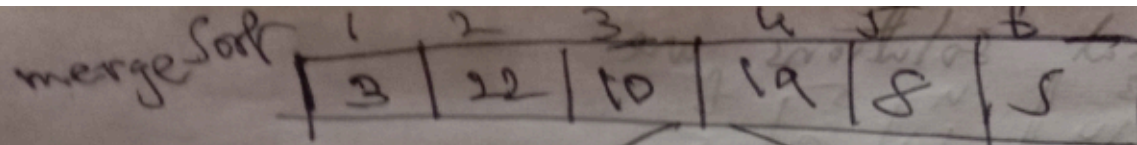
B:

1	2	3	4	5	6
3	5	8	10	19	22

Copy B into A

A:

3	5	8	10	19	22
---	---	---	----	----	----



MERGE SORT

```
Algorithm MergeSort(low, high)
{
    if(low < high)
    {
        mid :=  $\lfloor (low + high)/2 \rfloor$  ;
        MergeSort(low, mid);
        MergeSort(mid+1, high);
        Merge(low, mid, high);
    }
}
```

MERGE SORT

Algorithm Merge(low, mid, high)

```
{
  h := low, i := low, j := mid+1;
  while((h ≤ mid) and (j ≤ high))
  {
    if( a[h] ≤ a[j])
    {
      b[i] = a[h];
      h := h+1;
    }
    else
    {
      b[i] := a[j];
      j := j+1;
    }
    i := i+1;
  }
```

MERGE SORT

```
if(h > mid) then
  for k := j to high do
    {
      b[i] := a[k];
      i := i+1;
    }
else
  for k:= h to mid do
    {
      b[i] := a[k];
      i := i+1;
    }
  for k := low to high do
    a[k] := b[k];
}
```

MERGE SORT

$$T(n) = \begin{cases} a & \text{if } n = 1, a \text{ is constant} \\ 2T(n/2) + c n, & n > 1 \text{ and } c \text{ is constant} \end{cases}$$

- **When n is power of 2 $n = 2^i$**

$$\begin{aligned} T(n) &= 2 T(n/2) + c n \\ &= 2 [2T(n/4) + c n/2] + c n \\ &= 4 T(n/4) + 2 c n \\ &= 4 [2T(n/8) + c n/4] + 2 c n \\ &= 8 T(n/8) + 3 c n \\ &= 2^3 T(n/2^3) + 3 c n \\ &\cdot \\ &\cdot \\ &\cdot \\ &= 2^i T(n/2^i) + i c n \\ &= 2^i T(1) + i c n \end{aligned}$$

$$n = 2^i \text{ so } i = \log_2 n$$

$$T(n) = a n + c n \log_2 n$$

So the Best, Average and Worst case time complexity is **$T(n) = O(n \log_2 n)$**

QUICK SORT

- Quick sort is a fast sorting algorithm used to sort a list of elements. Quick sort algorithm is invented by **C. A. R. Hoare**.
- The quick sort algorithm attempts to separate the list of elements into two parts and then sort each part recursively.
- That means it use **divide and Conquer** strategy. In quick sort, the partition of the list is performed based on the element called ***pivot***. Here pivot element is one of the elements in the list.
- The list is divided into two partitions such that **"all elements to the left of pivot are smaller than the pivot and all elements to the right of pivot are greater than or equal to the pivot"**.

QUICK SORT

Algorithm QuickSort(p, q)

```
{  
    if(p < q) then  
    {  
        j := Partition(a, p, q+1);  
        QuickSort(p, j-1);  
        QuickSort(j+1, q);  
    }  
}
```

- The procedure **Partition()** takes the input as a problem and partitions the problem into two sub problems at the partitioning element j.
- Those two sub problems can be also solved using the same procedure.
- This procedure continues upto the total problem is solved completely.

QUICK SORT

- **Step by Step Process of Partition():**
- In Quick sort algorithm, partitioning of the list is performed using following steps...
- **Step 1** - Consider the first element of the list as pivot (i.e., Element at first position in the list).
- **Step 2** - Define two variables i and j . Set i and j to first and last elements of the list respectively.
- **Step 3** - Increment i until $\text{list}[i] > \text{pivot}$ then stop.
- **Step 4** - Decrement j until $\text{list}[j] < \text{pivot}$ then stop.
- **Step 5** - If $i < j$ then exchange $\text{list}[i]$ and $\text{list}[j]$.
- **Step 6** - Repeat steps 3,4 & 5 until $i > j$.
- **Step 7** - Exchange the pivot element with $\text{list}[j]$ element.

QUICK SORT

Algorithm Partition(a, m, p)

```
{
  v := a[m], i := m, j := p;
  repeat
  {
    repeat
      i := i+1;
    until (a[i] >= v);
    repeat
      j := j-1;
    until (a[j] <= v);
    if (i < j)
    {
      t := a[i];
      a[i] := a[j];
      a[j] := t;
    }
  } until (i >= j);
  a[m] := a[j];
  a[j] := v;
  return j;
}
```

QUICK SORT

Input: 65 70 75 80 60 55 50 45

Pivot:65 i

Pass1 65 70 75 80 60 55 50 45

(i) i j swap (A[i], A[j])

65 45 75 80 60 55 50 70

(ii) i j swap (A[i], A[j])

65 45 50 80 60 55 75 70

(iii) i j swap (A[i], A[j])

65 45 50 55 60 80 75 70

(iv) j i If(i>=j) break
swap (A[left], A[j])

60 45 50 55 65 80 75 70

Items <=65

Items >65

QUICK SORT

- **Time Complexity of Quick Sort :**

- A) **Worst Case :**

- The number of element comparisons in each call of partition is almost $p-m+1$ in $\text{Partition}(a, m, p)$.
 - Let r be the total number of elements in all the calls of partition at any level of recursion.
 - At level one only one call, $\text{Partition}(a, 1, n)$ is made and $r = n$.
 - At level 2 at most two calls are made and $r = n-1$ and so on.
 - At each level of recursion $O(r)$ element comparisons are made by the Partition procedure.
 - At each level r is at least one less than r at the previous level in the partition.
 - Hence the worst case time complexity ($C_w(n)$) is the sum of r as r varies from 2 to n .
 - $C_w(n) = 2 + 3 + \dots + n$ (Sum of n numbers)
 - $C_w(n) = n(n+1)/2 - 1 = (n^2 + n)/2 - 1$
 - So the worst-case time complexity is $T(n) = O(n^2)$.

QUICK SORT

- **B) Best Case :**

- In the best case the array can be divided into exactly two equal parts.
- So the time complexity is:

$$\begin{aligned}T(n) &= 2 T(n/2) + n \\&= 2 [2T(n/4) + n/2] + n \\&= 4 T(n/4) + 2 n \\&= 4 [2T(n/8) + n/4] + 2 n \\&= 8 T(n/8) + 3 n \\&= 2^3 T(n/2^3) + 3 n \\&\cdot \\&\cdot \\&\cdot \\&= 2^i T(n/2^i) + i n \\&= 2^i T(1) + i n\end{aligned}$$

$$n = 2^i \text{ so } i = \log_2 n$$

$$T(n) = a n + n \log_2 n$$

So the Best Case time complexity is **$T(n) = \Omega(n \log_2 n)$**

QUICK SORT

- **Average Case :**

- Under the assumption made earlier, the partitioning element v has an equal probability of being the i^{th} smallest element $1 \leq i \leq p - m$, in $a[m : p-1]$. Hence the two sub arrays to be sorted are $a[m : j]$ and $a[j+1 : p-1]$ with probability $1/(p-m)$, $m \leq j < p$. For this we obtain recurrence :
- $C_A(n) = (n+1) + 1/n \sum_{1 \leq k \leq n} [C_A(k-1) + C_A(n-k)] \dots\dots \text{eq-1.}$

The number of element comparisons required by partition on its first call are $n+1$. note that $C_A(0) = C_A(1) = 0$. Multiply both the sides of eq-1 by n

$$n C_A(n) = n (n+1) + 2 [C_A(0) + C_A(1) + \dots + C_A(n-1)] \dots\dots \text{eq-2}$$

Replace n by $n-1$ in eq-2 gives

$$(n - 1) C_A(n-1) = n (n-1) + 2 [C_A(0) + C_A(1) + \dots + C_A(n-2)] \dots\dots \text{eq-3}$$

Subtract eq-3 from eq-2

$$n C_A(n) - (n - 1) C_A(n-1) = 2n + 2 C_A(n-1)$$

$$C_A(n)/n+1 = C_A(n-1)/n + 2/n+1$$

Repeatedly using the equation to substitute for $C_A(n-1)$, $C_A(n-2)$ We get

$$\begin{aligned} C_A(n) / n+1 &= C_A(n-2)/n-1 + 2/n + 2/n+1 \\ &= C_A(n-3)/n-2 + 2/n-1 + 2/n + 2/n+1 \end{aligned}$$

•

•

$$= C_A(1)/2 + 2 \sum_{3 < k \leq n+1} 1/k$$

QUICK SORT

- $$C_A(n) / (n+1) = 2 \sum_{3 < k \leq n+1} 1/k$$

since $\sum_{3 < k \leq n+1} 1/k \leq \int_2^{n+1} \frac{1}{x} dx$
 $= \log_e(n+1) - \log_e 2$

$$C_A(n) \leq 2(n+1) [\log_e(n+1) - \log_e 2] = O(n \log_e n)$$

STRASSEN'S MATRIX MULTIPLICATION

- In linear algebra, the **Strassen** algorithm, named after **Volker Strassen**, is an algorithm for **matrix multiplication**. It is faster than the standard **matrix**

```
void multiply(int A[][N], int B[][N], int C[][N])
```

```
{  
    for (int i = 0; i < N; i++)  
    {  
        for (int j = 0; j < N; j++)  
        {  
            C[i][j] = 0;  
            for (int k = 0; k < N; k++)  
            {  
                C[i][j] += A[i][k]*B[k][j];  
            }  
        }  
    }  
}
```

Time Complexity of above method is $O(N^3)$.

STRASSEN's MATRIX MULTIPLICATION

- **Divide and Conquer :**

Following is simple Divide and Conquer method to multiply two square matrices.

1) Divide matrices A and B in 4 sub-matrices of size $N/2 \times N/2$ as shown in the below diagram.

2) Calculate following values recursively. $ae + bg$, $af + bh$, $ce + dg$ and $cf + dh$.

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \times \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{bmatrix}$$

$A \qquad \qquad \qquad B \qquad \qquad \qquad C$

A, B and C are square matrices of size $N \times N$

a, b, c and d are submatrices of A, of size $N/2 \times N/2$

e, f, g and h are submatrices of B, of size $N/2 \times N/2$

In the above method, we do 8 multiplications for matrices of size $N/2 \times N/2$ and 4 additions. Addition of two matrices takes $O(N^2)$ time.

So the time complexity can be written as $T(N) = 8T(N/2) + O(N^2)$ From [Master's Theorem](#), time complexity of above method is $O(N^3)$ which is unfortunately same as the above naive method.

STRASSEN'S MATRIX MULTIPLICATION

● Strassen's Matrix Multiplication Algorithm:

- In this context, using Strassen's Matrix multiplication algorithm, the time consumption can be improved a little bit.
- Strassen's Matrix multiplication can be performed only on **square matrices** where **n** is a **power of 2**. Order of both of the matrices are **n × n**.
- Divide **X**, **Y** and **Z** into four $(n/2) \times (n/2)$ matrices as represented below –

$$X = \begin{matrix} A & B \\ C & D \end{matrix} \quad Y = \begin{matrix} E & F \\ G & H \end{matrix} \quad Z = \begin{matrix} I & J \\ K & L \end{matrix}$$

Using Strassen's Algorithm compute the following –

- $M1 := (A+C) \times (E+F)$
- $M2 := (B+D) \times (G+H)$
- $M3 := (A-D) \times (E+H)$
- $M4 := A \times (F-H)$
- $M5 := (C+D) \times (E)$
- $M6 := (A+B) \times (H)$
- $M7 := D \times (G-E)$

STRASSEN's MATRIX MULTIPLICATION

- Then,
 - $I := M_2 + M_3 - M_6 - M_7$
 - $J := M_4 + M_6$
 - $K := M_5 + M_7$
 - $L := M_1 - M_3 - M_4 - M$
 - Analysis
 - $T(n) = \begin{cases} c & \text{if } n=1 \\ 7 \times T(n/2) + d \times n^2 & \text{otherwise} \end{cases}$
- where c and d are constants
- Using this recurrence relation, we get $T(n) = O(n^{\log 7})$
 - Hence, the complexity of Strassen's matrix multiplication algorithm is $O(n^{\log 7})$.
Which is equal to $O(n^{2.81})$